```
import Database from 'better-sqlite3';
import { fileURLToPath } from 'url';
import { dirname, join } from 'path';

const __dirname = dirname(fileURLToPath(import.meta.url));
const DB_PATH = join(__dirname, '..', 'data', 'jobs.db');

let db;

export function initDB() {
  db = new Database(DB_PATH);
  db.pragma('journal_mode = WAL');

  db.exec(`
    CREATE TABLE IF NOT EXISTS jobs (
      id TEXT PRIMARY KEY,
      url TEXT NOT NULL,
      title TEXT,
      description TEXT,
      budget_type TEXT,
      budget_min REAL,
      budget_max REAL,
      posted_at TEXT,
      seen_at TEXT DEFAULT CURRENT_TIMESTAMP,
      match_score REAL,
      match_reasoning TEXT,
      category TEXT,
      status TEXT DEFAULT 'pending',
      cover_letter TEXT,
      applied_at TEXT,
      telegram_msg_id INTEGER
    );

    CREATE INDEX IF NOT EXISTS idx_status ON jobs(status);
    CREATE INDEX IF NOT EXISTS idx_seen ON jobs(seen_at);
  `);

  return db;
}

export function getDB() {
  if (!db) initDB();
  return db;
}
```

```
export function jobExists(id) {
  const row = getDB().prepare('SELECT id FROM jobs WHERE id = ?').get(id);
  return !!row;
}

export function insertJob(job) {
  const stmt = getDB().prepare(`
    INSERT OR IGNORE INTO jobs
    (id, url, title, description, budget_type, budget_min, budget_max, posted_at,
    VALUES (@id, @url, @title, @description, @budget_type, @budget_min, @budget_m
  `);
  return stmt.run(job);
}

export function updateJobStatus(id, status, extra = {}) {
  const fields = ['status = ?'];
  const values = [status];

  if (extra.telegramMsgId) {
    fields.push('telegram_msg_id = ?');
    values.push(extra.telegramMsgId);
  }
  if (status === 'applied') {
    fields.push('applied_at = CURRENT_TIMESTAMP');
  }

  values.push(id);
  getDB().prepare(`UPDATE jobs SET ${fields.join(', ')} WHERE id = ?`).run(...val
}

export function getJobById(id) {
  return getDB().prepare('SELECT * FROM jobs WHERE id = ?').get(id);
}

export function getStats() {
  return getDB().prepare(`
    SELECT
      COUNT(*) as total_seen,
      SUM(CASE WHEN status = 'applied' THEN 1 ELSE 0 END) as applied,
      SUM(CASE WHEN status = 'pending' THEN 1 ELSE 0 END) as pending,
      SUM(CASE WHEN status = 'skipped' THEN 1 ELSE 0 END) as skipped,
      SUM(CASE WHEN status = 'rejected' THEN 1 ELSE 0 END) as rejected,
      AVG(match_score) as avg_match
    FROM jobs
  `).get();
}
```